

计算机组成与体系结构

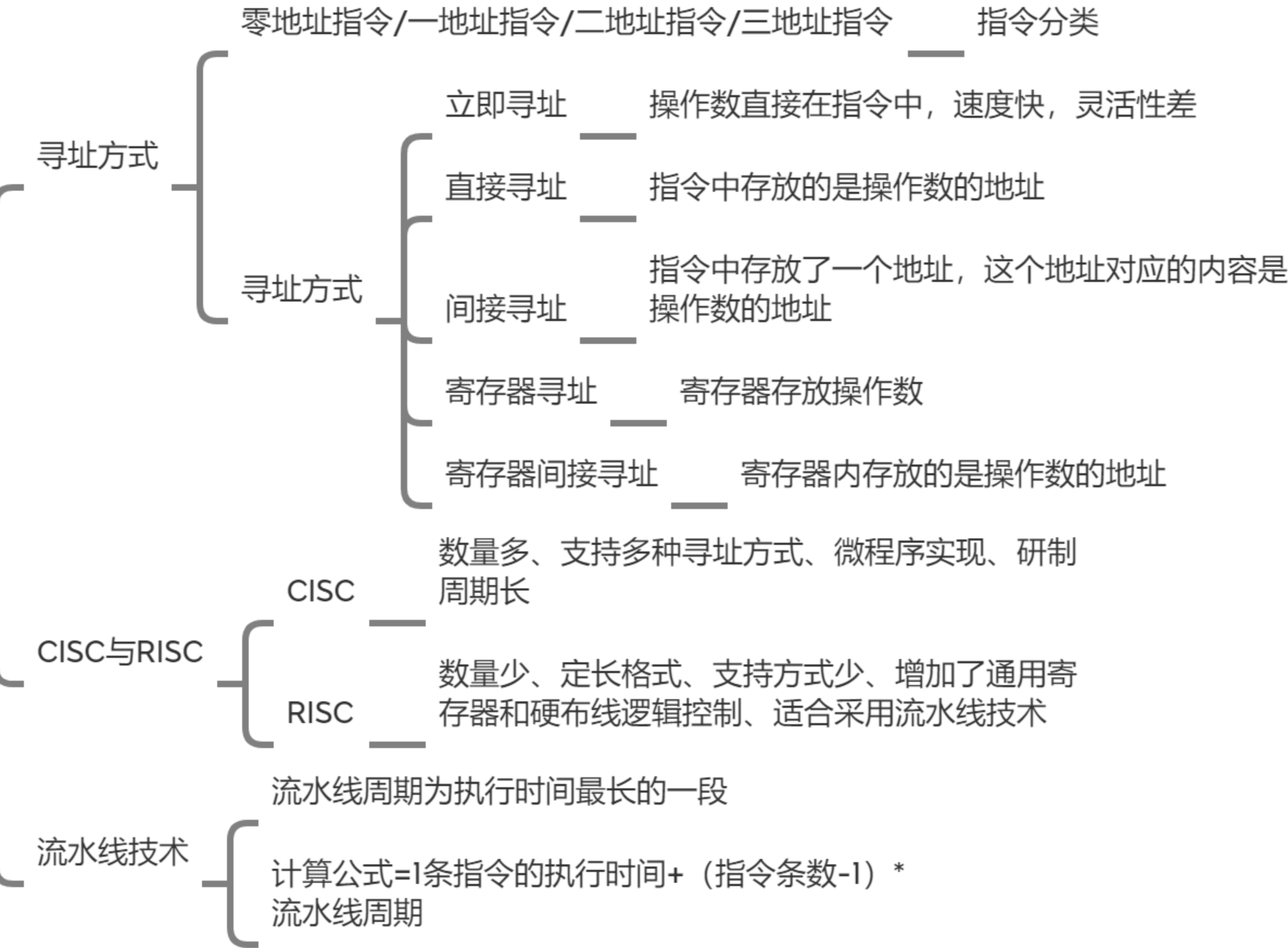
计算机基础知识



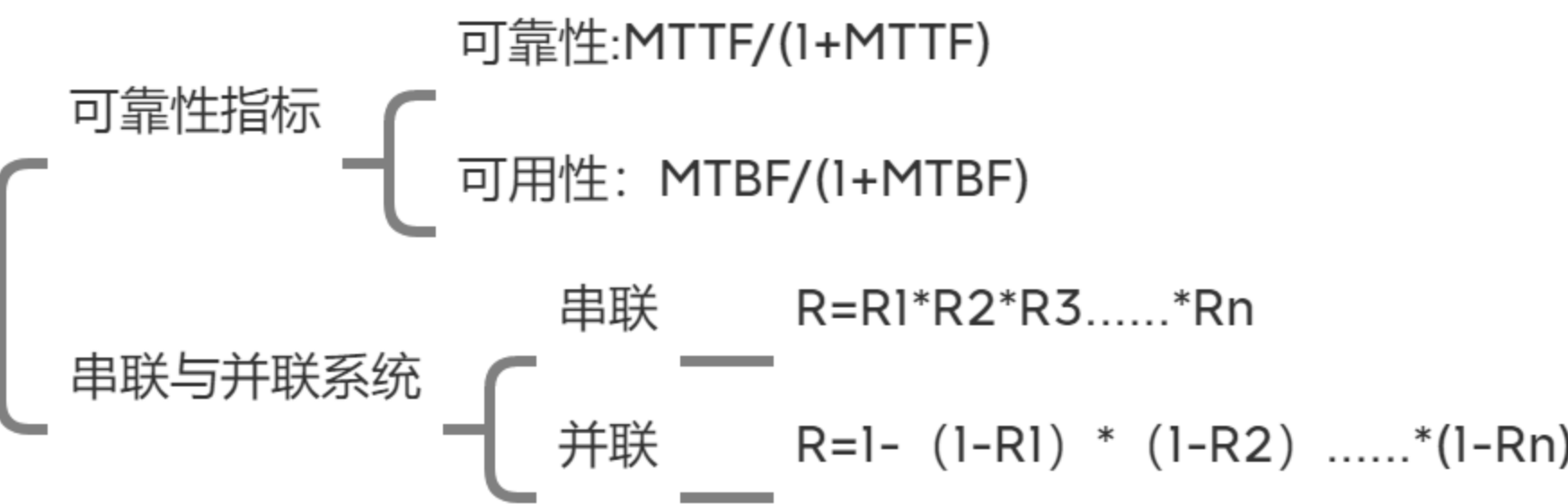
计算机组成



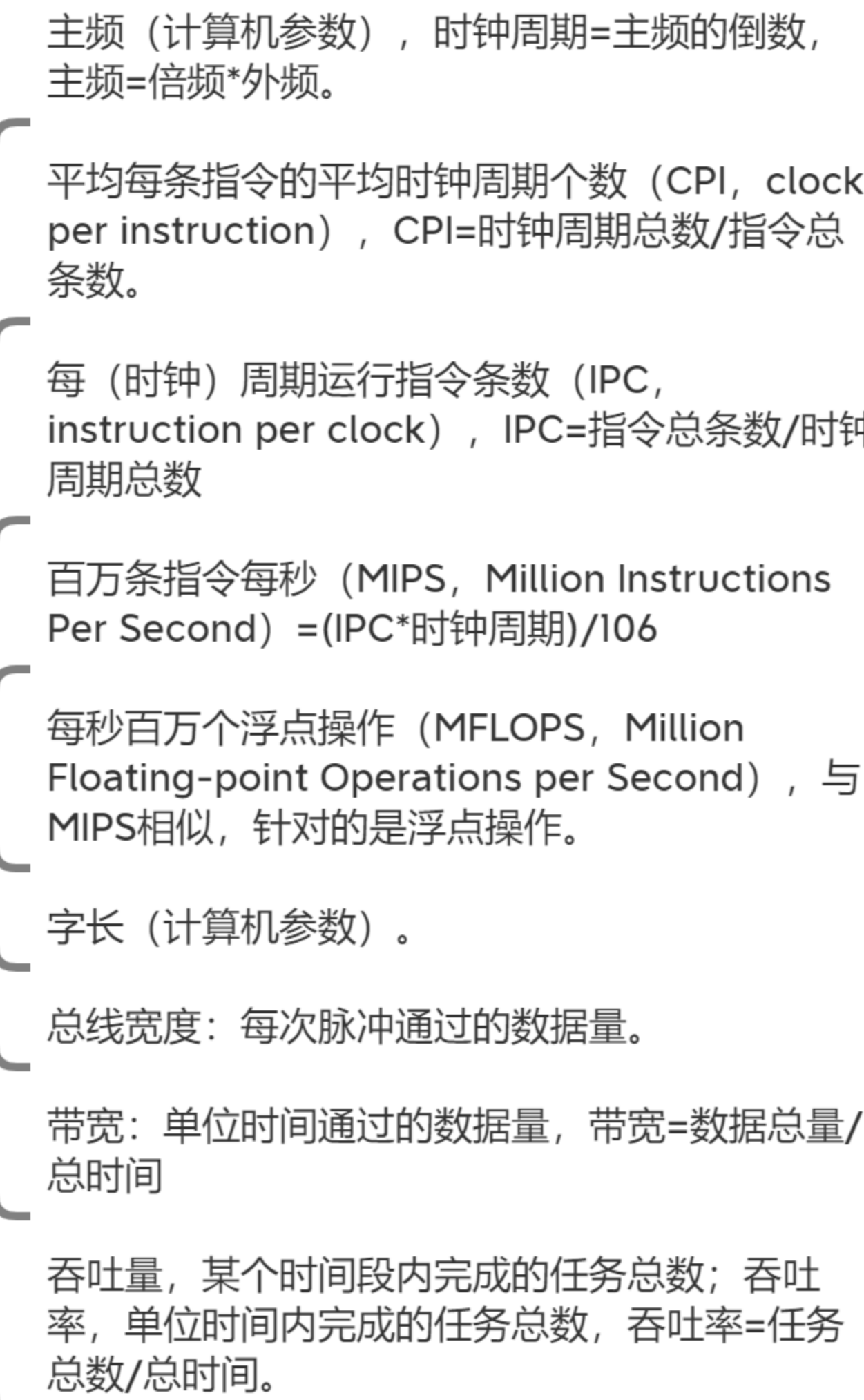
指令系统



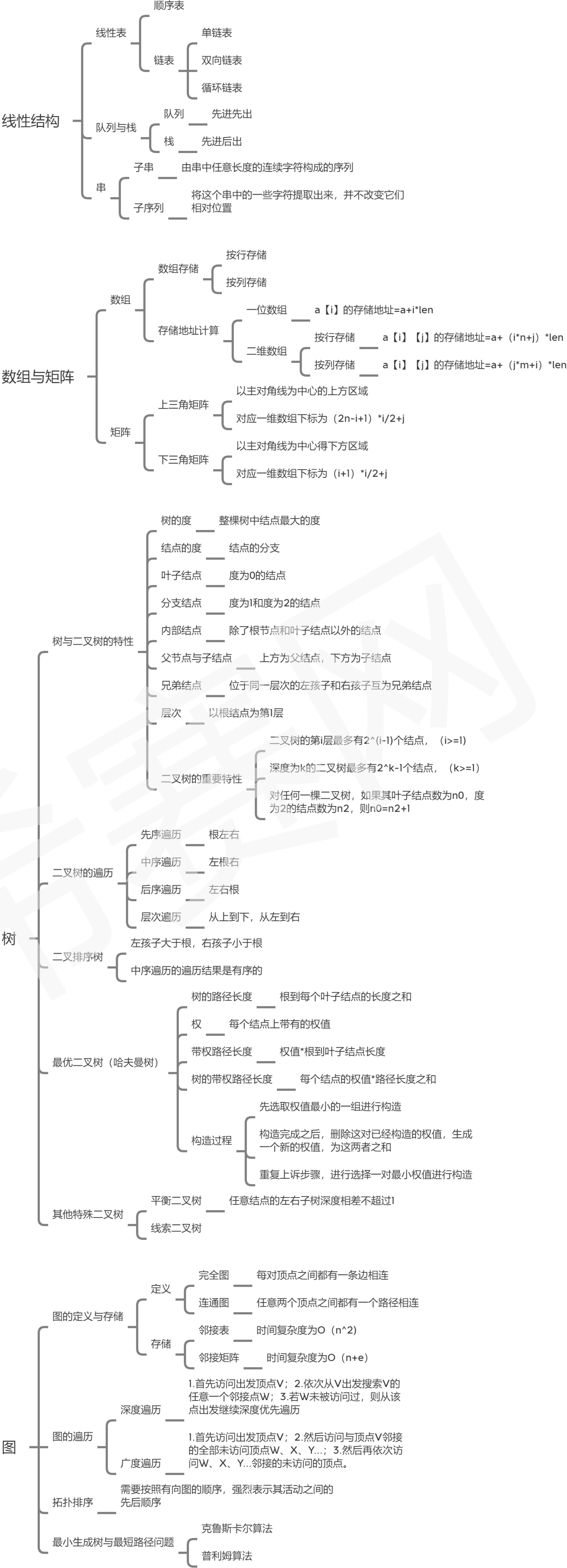
可靠性



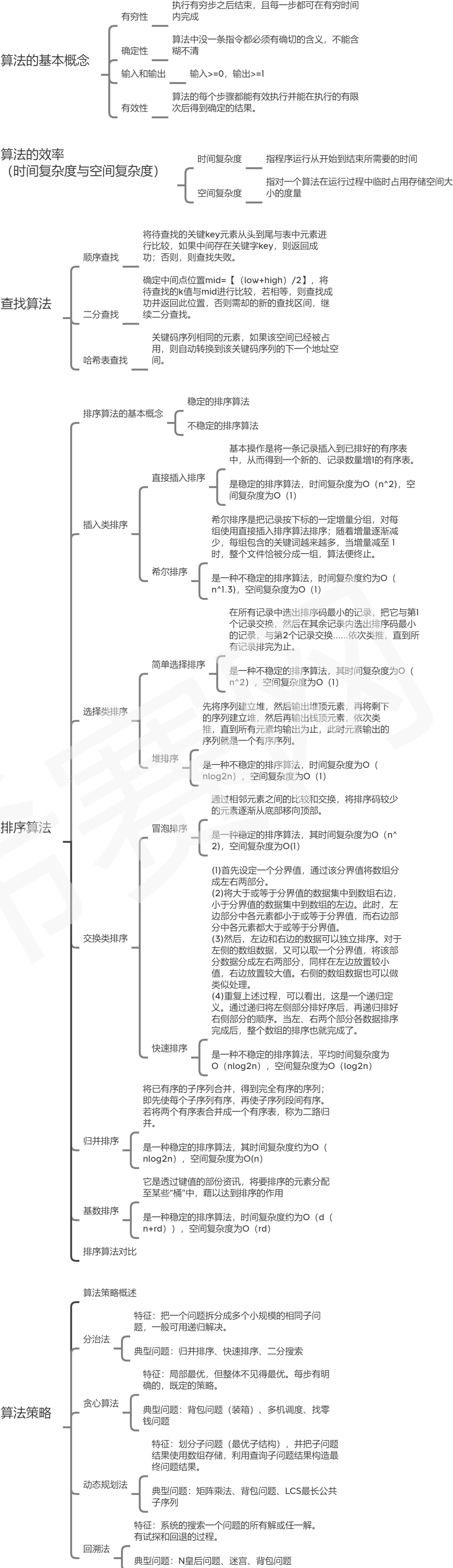
计算机性能指标



数据结构



算法基础



程序设计语言基础与语言处理程序

程序设计语言概述

编译程序与解释程序

编译型语言

翻译程序：编译器，生成目标代码、目标程序直接执行、编译器不参与执行、执行效率高、灵活性差，可移植性差。

解释型语言

翻译程序：解释器，不会生成目标代码、边解释边执行、解释器参与执行、执行效率低、灵活性好，可移植性强。

常见程序设计语言的特点

Fortran语言（科学计算，执行效率高）

Pascal语言（为教学而开发的，表达能力强，Delphi）

C语言（指针操作能力强，高效）

Lisp语言（函数式程序语言，符号处理，人工智能）

C++语言（面向对象，高效）

JAVA语言（面向对象，中间代码，跨平台）

C#语言（面向对象，中间代码，.Net）

Prolog语言（逻辑推理，简洁性，表达能力，数据库和专家系统）

Python语言（解释型，面向对象，胶水语言）

语言分类

- 解释型（逐句，解释器，跨平台，执行效率低）
- 编译型（逐段编译，生成可执行程序exe等，执行效率高）

程序设计语言的基本成分

数据类型

运算

控制结构

顺序结构

选择结构

循环结构

函数调用方式

传值调用

形参取的是实参的值，形参的改变不会导致调用点所传的实参的值发生改变

引用调用

形参取的是实参的地址，即相当于实参存储单元的地址引用，因此其值的改变同时就改变了实参的值。

编译过程概述

词法分析

从左到右逐个扫描源程序中的字符，识别其中如关键字、标识符、常数、运算符以及分隔符等。

语法分析

根据语法规则将单词符号分解成各类语法单位，并分析源程序是否存在语法上的错误。

语义分析

进行类型分析和检查，主要检测源程序是否存在静态语义错误。

中间代码

根据语义分析的输出生成中间代码。

错误管理

动态错误

发生在程序运行时

静态错误

编译时所发现的程序错误

符号表

符号表的作用是记录源程序中各个符号的必要信息，以辅助语义的正确性检查和代码生成，在编译过程中需要对符号表进行快速有效地查找、插入、修改和删除等操作。

文法

0型：短语文法

典型代表：图灵机

1型上下文有关文法

线性界限自动机

2型上下文无关文法

非确定的下推自动机

3型：正规文法

有限自动机

正规式与正规集

ab 字符串ab构成的集合

a|b 字符串a、b构成的集合

a* 由0或多个a构成的字符串集合

(a|b)*所有字符a和b构成的串的集合

有限自动机

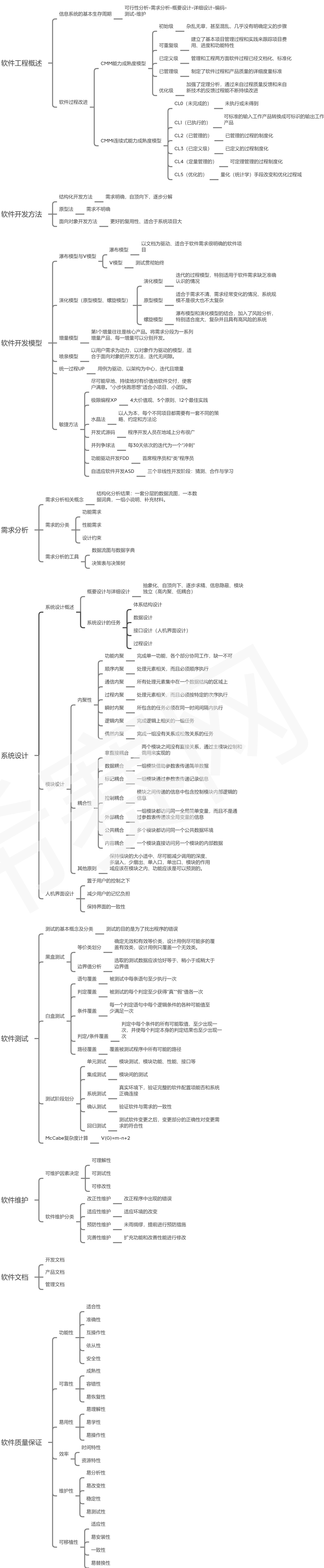
表达式

前缀表达式

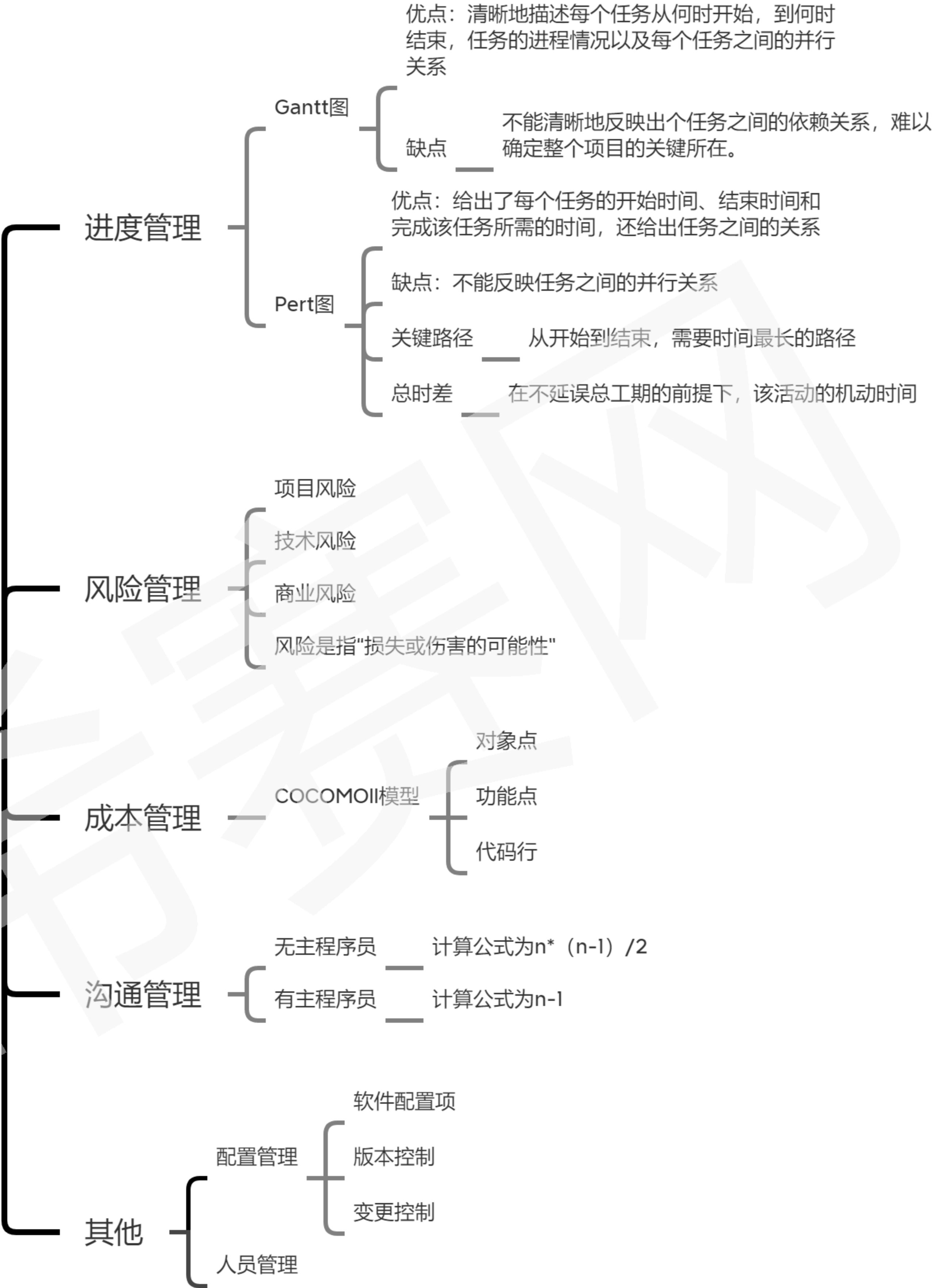
中缀表达式

后缀表达式（逆波兰式）

系统开发基础



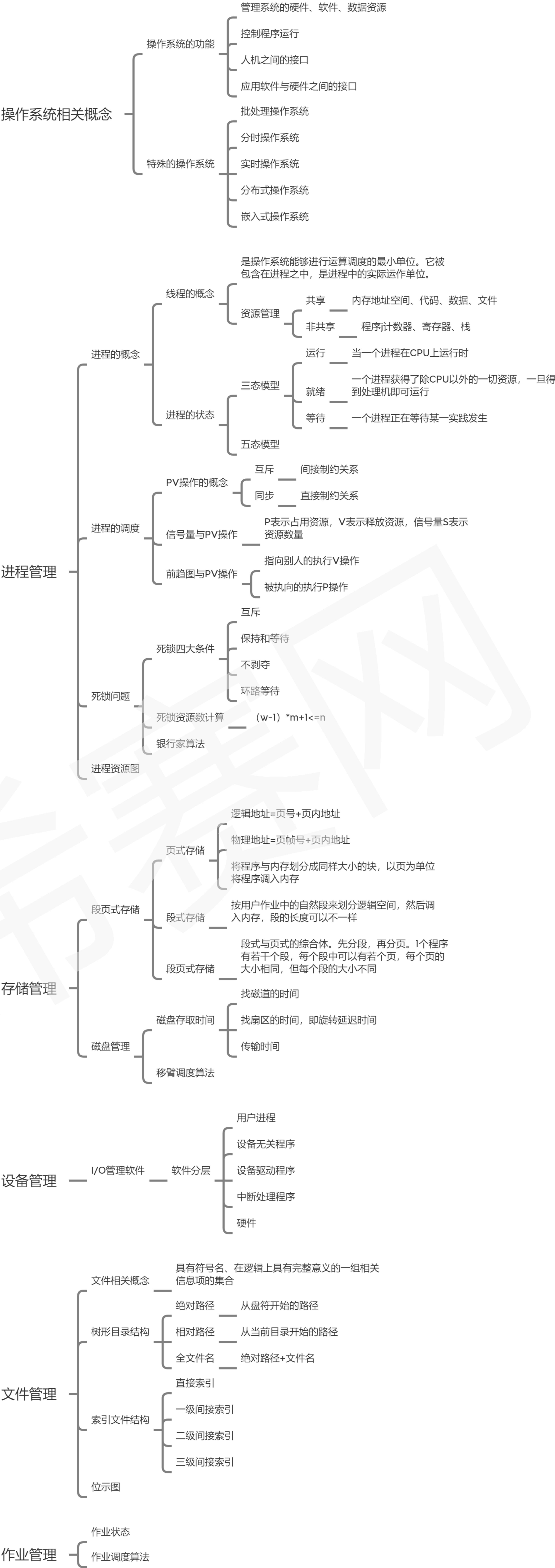
项目管理



面向对象技术

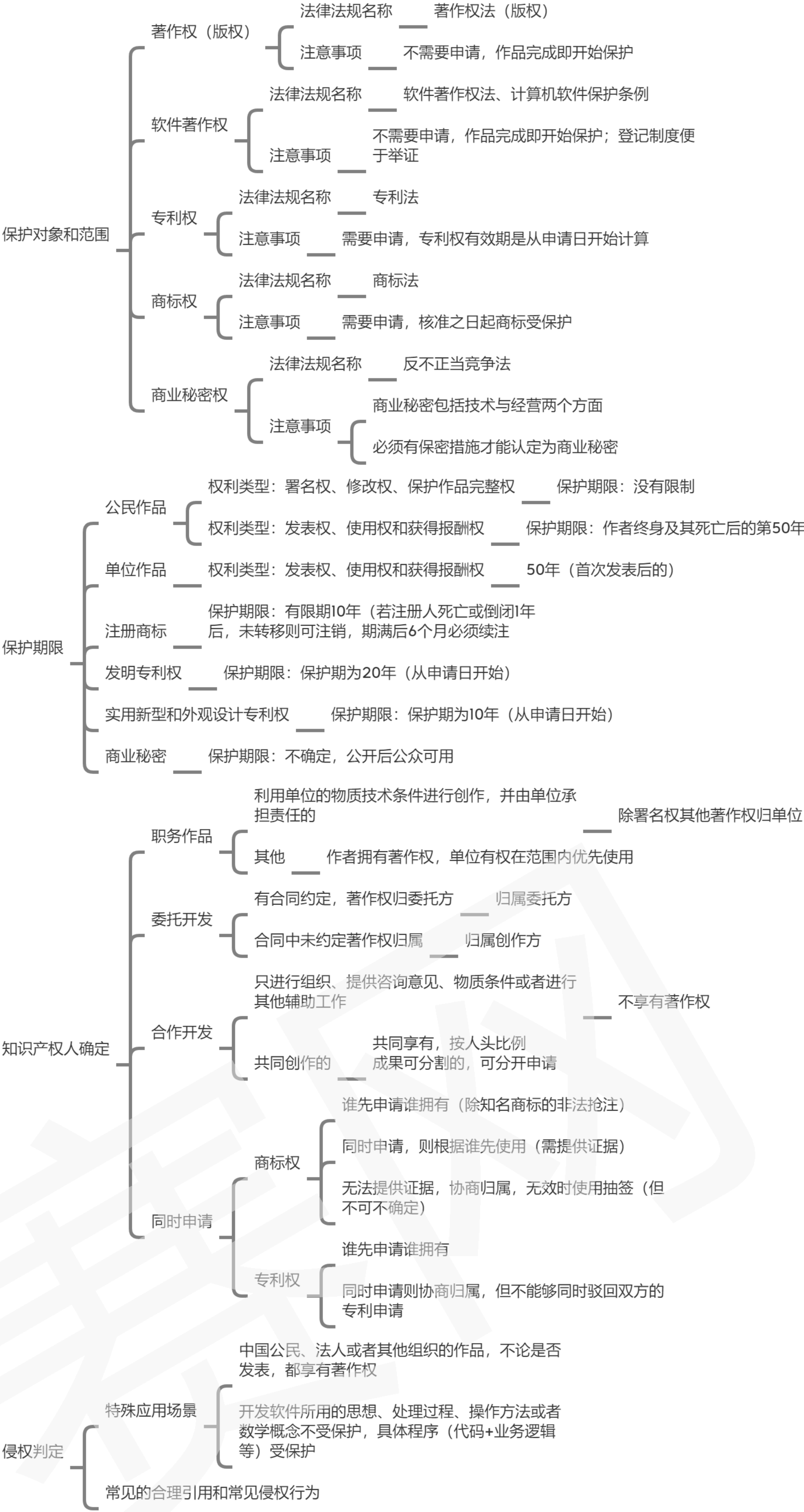


操作系统



知识产权与标准化

知识产权

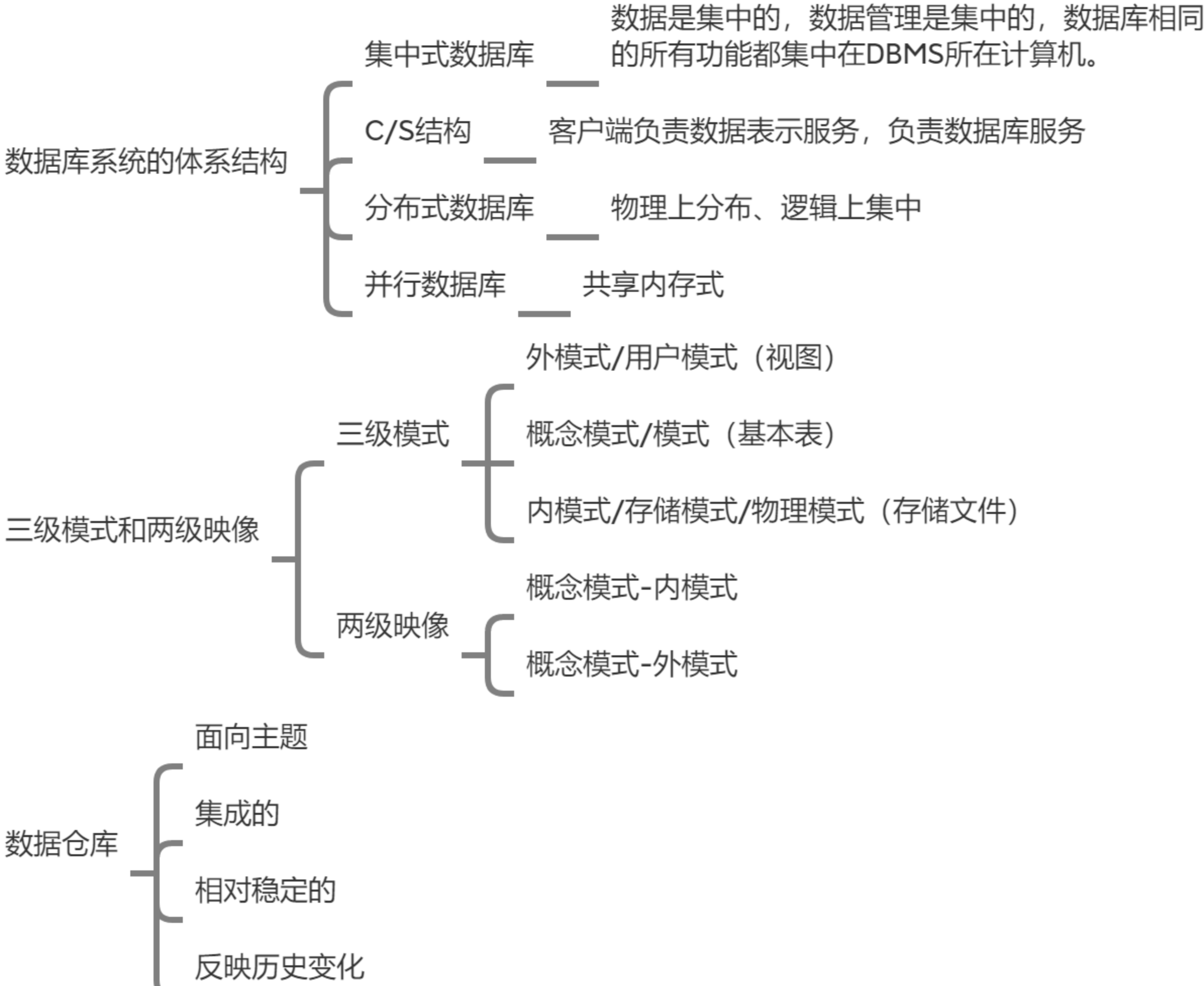


标准化

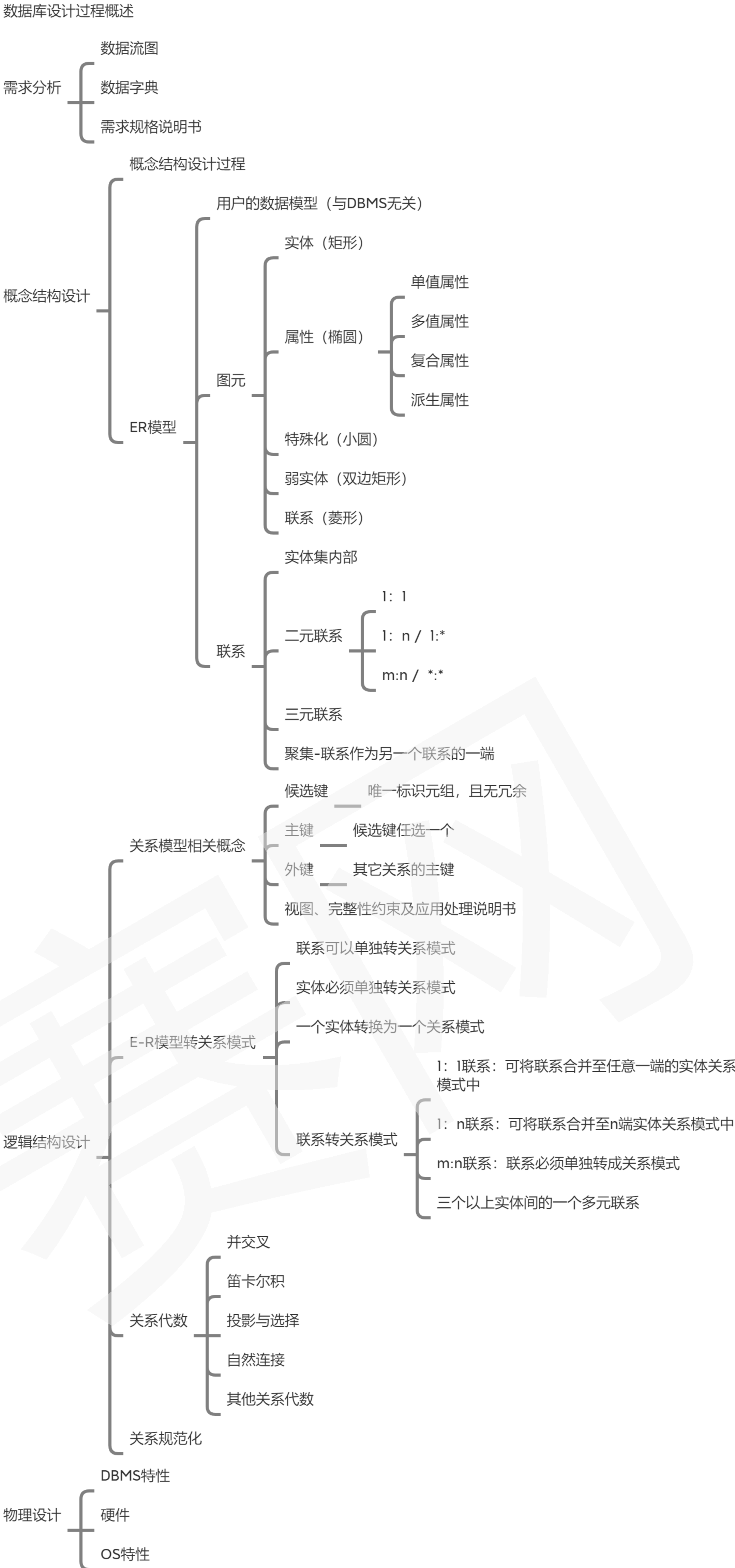


数据库系统

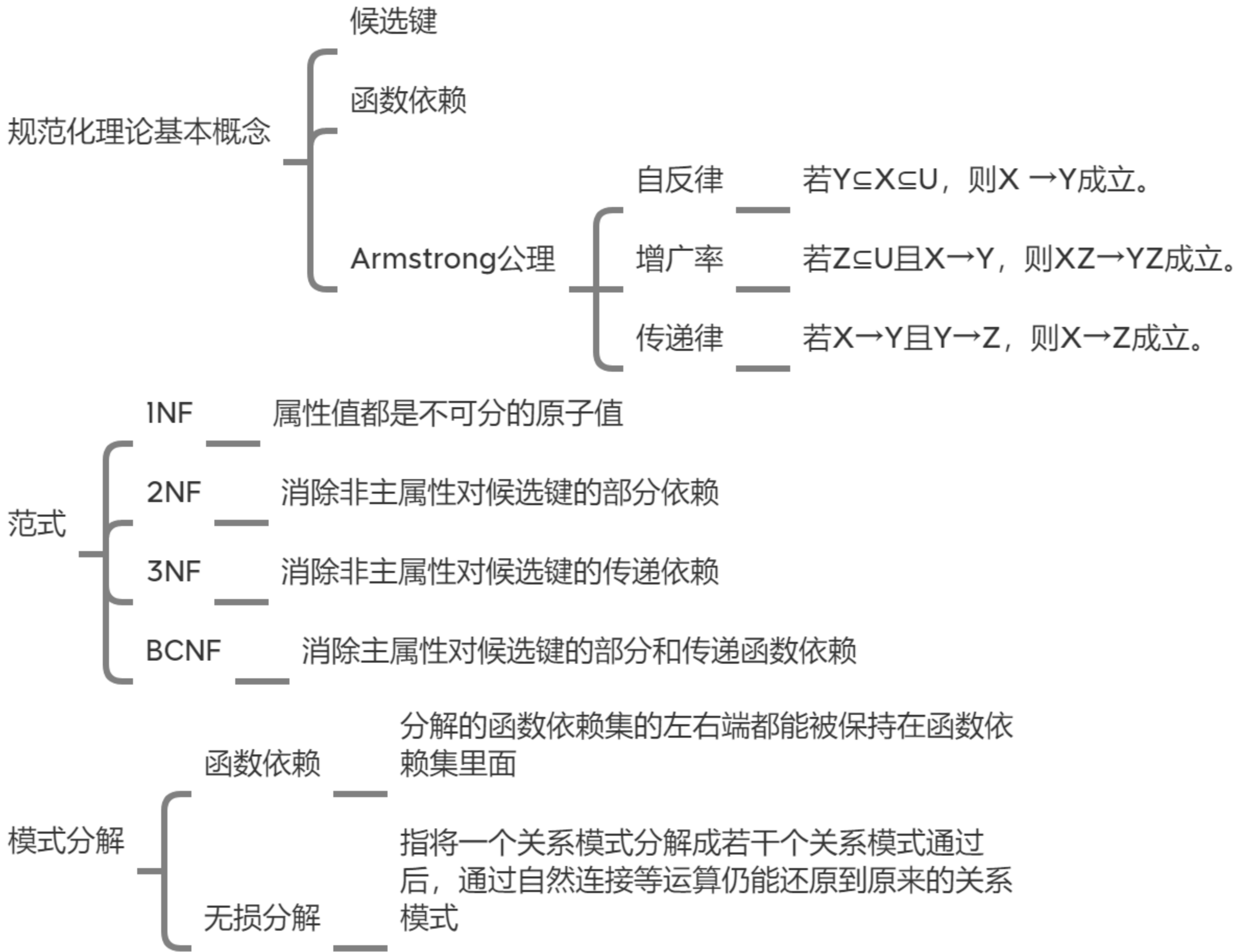
数据库的基本概念



数据库设计过程



规范化理论



SQL语言

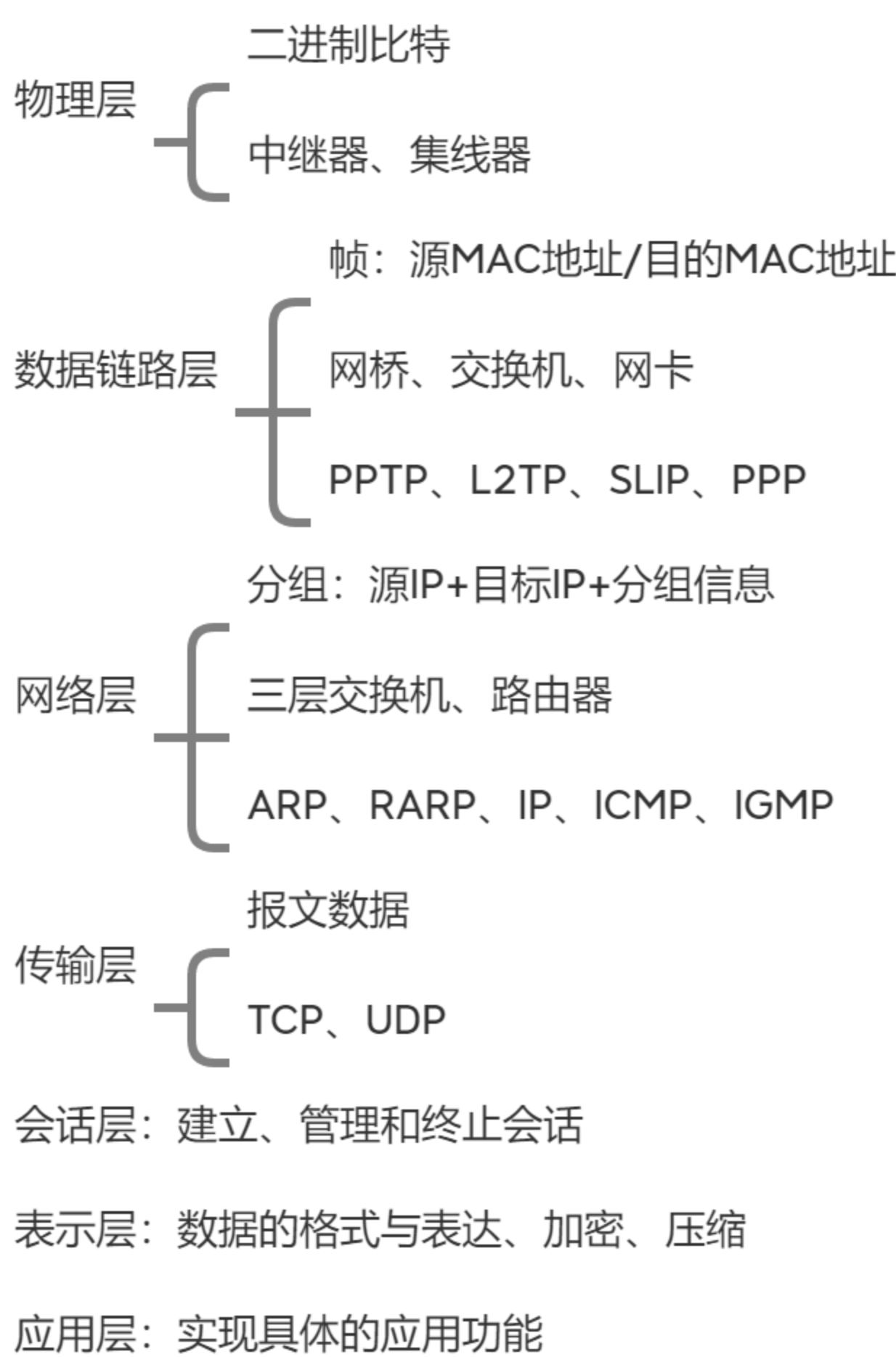


并发控制

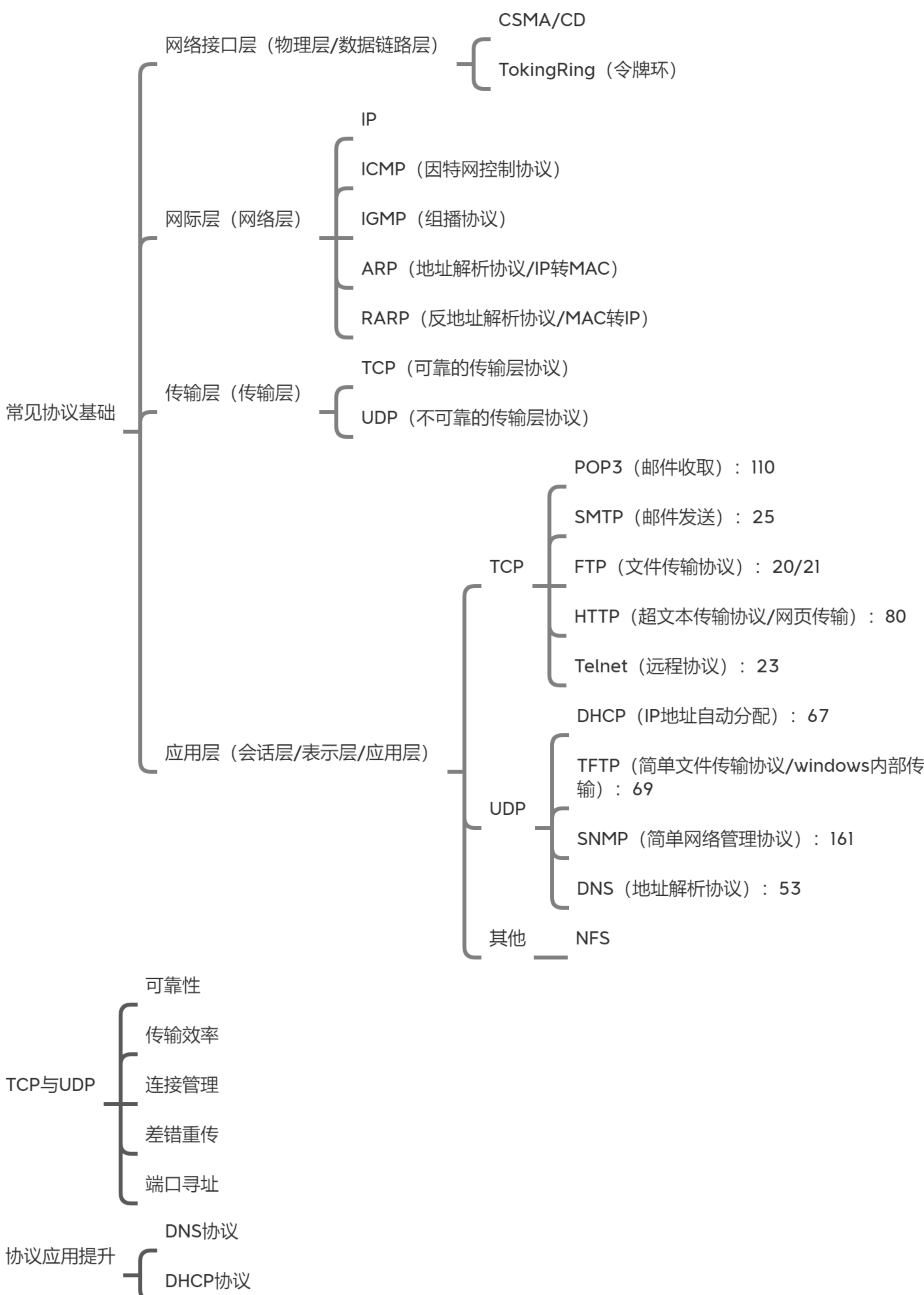


计算机网络

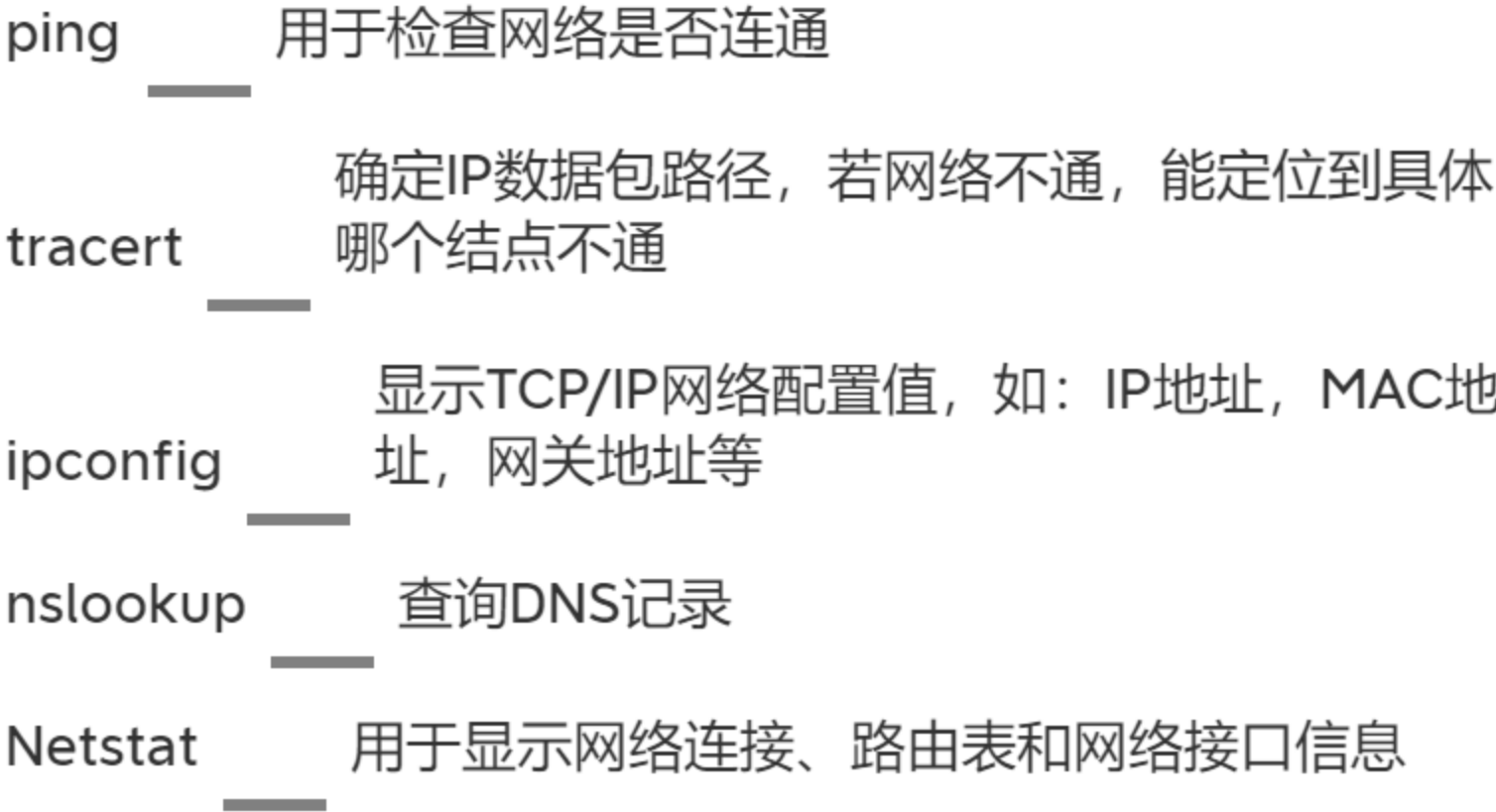
OSI/RM七层模型



TCP/IP协议簇



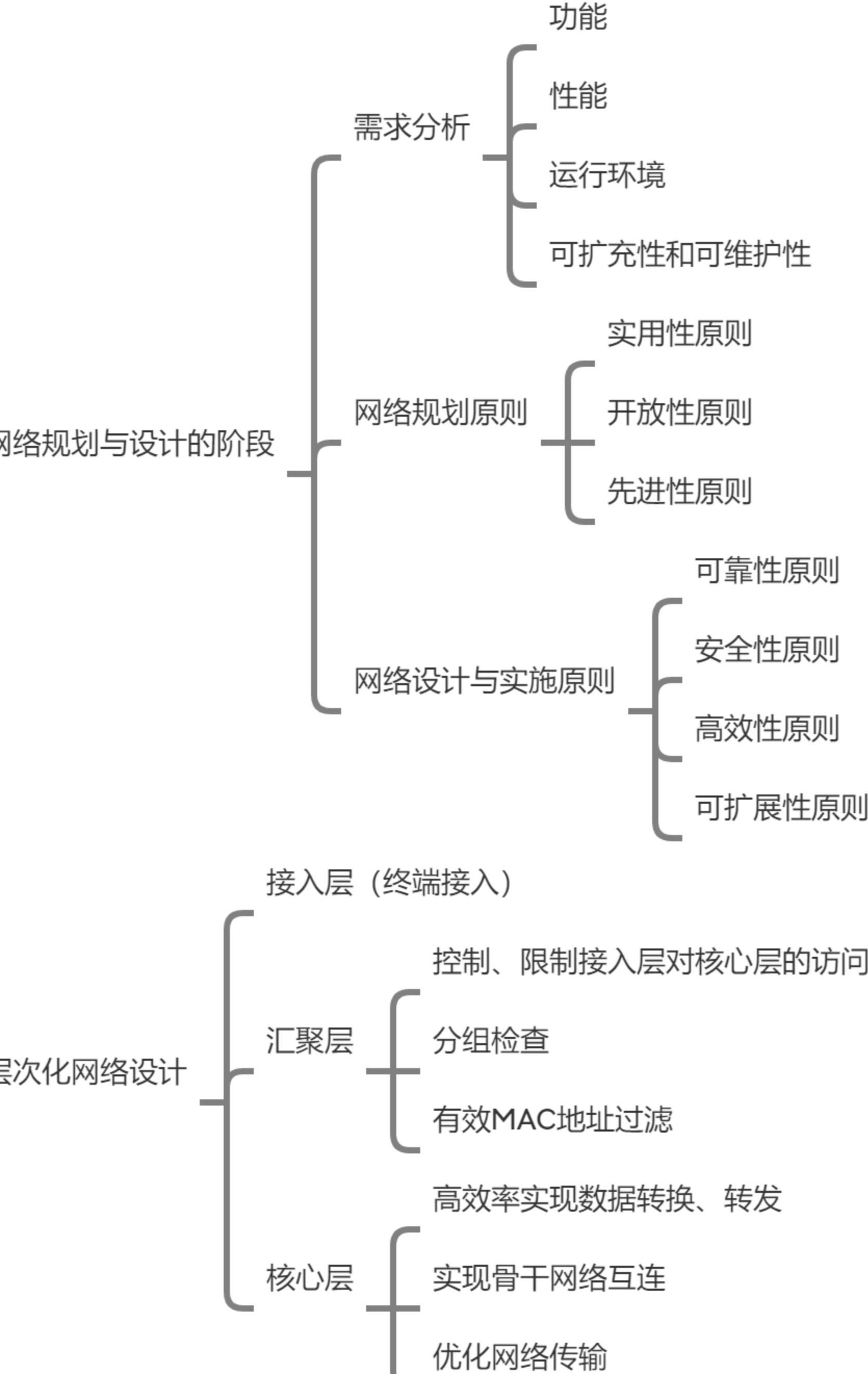
网络诊断命令



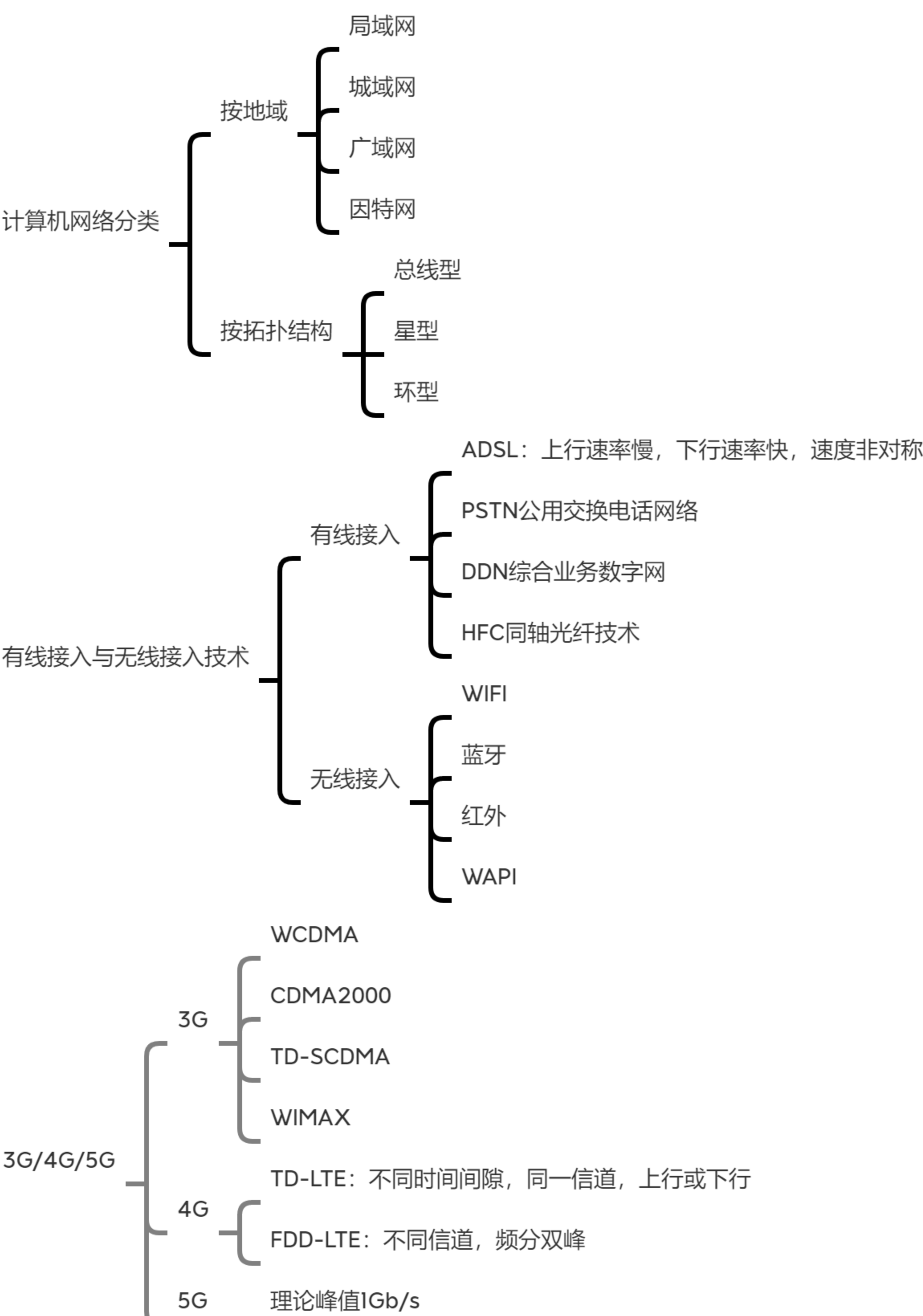
IP地址与子网划分



网络规划与设计



网络接入技术



WWW服务



信息安全

